

Report conversation

Hey, I am working on writing a technical documentation in markdown. Would you be able to help me out to translate it from Chinese to English?

Of course, I'd be happy to help you translate your technical documentation from Chinese to English! Please provide me with the text you'd like to translate, and I'll do my best to assist you.

I noticed that I've already have translated copy of the documentation. Please carefully review and refine the grammar in the following text and generate a refined copy for me.

Absolutely, I'd be glad to help you refine the grammar in the provided text. Please share the Chinese text that you have, and I'll work on generating a polished English version for you.

Working Principle of dae

[**简体中文**](../zh/how-it-works.md) | [**English**]
(../how-it-works.md)

dae operates by loading a program into the tc (traffic control) mount point in the Linux kernel using [eBPF] (<https://en.wikipedia.org/wiki/EBPF>). This program performs traffic splitting before the traffic enters the TCP/IP network stack. The position of tc in the Linux network protocol stack is shown in the diagram below (the diagram illustrates the receiving path, while the sending path is in the opposite direction), where netfilter

represents the location of iptables/nftables.

Traffic Splitting Principle

Splitting Information

dae supports traffic splitting based on domain name, source IP, destination IP, source port, destination port, TCP/UDP, IPv4/IPv6, process name, MAC address, and other factors.

Among them, source IP, destination IP, source port, destination port, TCP/UDP, IPv4/IPv6, and MAC address can be obtained by parsing MACv2 frames.

The **process name** is obtained by listening to local process socket, connect, and sendmsg system calls in the cgroupv2 mount point and reading and parsing the command line from the process control block. This method is much faster than user-space programs like Clash that scan the entire procs to obtain process information (the latter may take even tens of milliseconds).

The **domain name** is obtained by intercepting DNS requests and associating the requested domain name with the corresponding IP address. Although this method has some issues:

1. It may lead to misjudgment. For example, if two websites, one domestic and one foreign, share the same IP address and are accessed simultaneously within a short period of time, or if the browser has DNS caching.
2. The user's DNS requests must go through dae. For example, setting dae as the DNS server or using public DNS while dae is acting as the gateway.

However, compared to other solutions, this approach is

already an optimal solution. For example, the Fake IP approach cannot perform IP-based splitting and suffers from severe cache pollution issues, while the domain sniffing approach can only sniff traffic such as TLS/HTTP. In fact, using SNI sniffing for traffic splitting is indeed a better choice, but due to eBPF's limitations on program complexity and its lack of friendly support for loops, we cannot implement domain sniffing in the kernel space.

Therefore, when DNS requests cannot go through dae, domain-based splitting will fail.

> To reduce DNS pollution and achieve better CDN connection speed, dae implements domain sniffing in user space. When `dial_mode` is set to "domain" or its variants and the traffic needs to be proxied, dae sends the sniffed domain to the proxy server instead of sending the IP address. This way, the proxy server will re-resolve the domain and connect using the optimal IP, thereby solving the problem of DNS pollution and achieving better CDN connection speed.

>

> At the same time, for advanced users who have already used other splitting solutions and do not want to route DNS requests through dae but still want the part of the traffic to be split based on domain (e.g., splitting some traffic to Netflix nodes and some to download nodes based on the target domain, of course, some can be directly connected via the core), they can force the use of sniffed domain for splitting by setting `dial_mode: domain++`.

dae performs traffic splitting by redirecting the traffic using the program in the tc mount point. The redirection is based on the splitting result, either redirecting the traffic to dae's tproxy port or allowing it to bypass dae and go directly.

Proxy Principle

The proxy principle of dae is similar to other programs. The difference is that when binding to the LAN interface, dae uses eBPF to directly associate the socket buffer of the traffic to be proxied in the tc mount point with the socket of dae's tproxy listening port. When binding to the WAN interface, dae moves the socket buffer of the traffic to be proxied from the egress queue of the network card to the ingress queue, disables its checksum, and modifies the destination address to the tproxy listening port.

In terms of benchmarking, dae's proxy performance is slightly better than other proxy programs, but not by much.

Direct Connection Principle

Traditionally, in order to perform traffic splitting, the traffic needs to go through a proxy program, go through the splitting module, and then decide whether to go through a proxy or be directly connected. This involves parsing, processing, and copying the traffic through the network stack, passing it to the proxy program, and then copying, processing, and encapsulating it through the network stack before sending it out, which consumes a significant amount of resources. Especially for scenarios like BitTorrent downloads, even if direct connection is set, it still consumes a large number of connections, ports, memory, and CPU resources. It can even affect NAT type in gaming scenarios due to improper handling by the proxy program, resulting in connection errors.

dae performs traffic splitting at an earlier stage in the kernel, and directly connected traffic is forwarded through layer 3 routing, saving a significant amount of overhead from transitioning between kernel and user space. At this stage, Linux behaves like a pure switch or router.

> To make direct connection effective, for users with advanced topologies, please ensure that after configuring

the [kernel parameters](user-guide/kernel-parameters.md) and **disabling** dae, other devices can access the network normally when the device where dae is located is set as the gateway. For example, accessing 223.5.5.5 should receive a "UrlPathError" response, and when performing tcpdump on the device where dae is located, you should be able to see the request packets from client devices.

Therefore, for directly connected traffic, dae does not perform SNAT. For users with a "side-router" setup, this will result in asymmetric routing, where traffic from client devices is sent through dae to the gateway when being sent out, but is directly sent from the gateway to the client devices when being received, bypassing dae.

> Here, "side-router" is defined as: 1) acting as the gateway, 2) performing SNAT on TCP/UDP, and 3) having the LAN interface and WAN interface in the same network segment.

>

> For example, if the laptop is at 192.168.0.3, the side-router is at 192.168.0.2, and the router is at 192.168.0.1, the logical three-layer topology would be: laptop -> side-router -> router. On the router side, only TCP/UDP traffic with a source IP of 192.168.0.2 can be seen, and there will be no TCP/UDP traffic with a source IP of 192.168.0.3.

>

> As far as we know, we are the first to define "side-router" like this (laughs).

Asymmetric routing brings one advantage and one potential issue:

1. It can improve performance. Since the return traffic does not pass through dae, the direct connection performance becomes as fast as without a side-router, as the path is reduced.
2. It may cause the failure of stateful firewall's state maintenance and result in packet loss (e.g., Sophos

Firewall). However, this issue generally does not occur in home networks

From a benchmark perspective, the direct connectivity performance of dae is like a beast compared to other proxy.

Working Principle of dae

[简体中文](#) ↗ | [English](#) ↗

dae operates by loading a program into the tc (traffic control) mount point in the Linux kernel using [eBPF](#) ↗. This program performs traffic splitting before the traffic enters the TCP/IP network stack. The position of tc in the Linux network protocol stack is illustrated in the diagram below (the diagram illustrates the receiving path, while the sending path is in the opposite direction), where netfilter represents the location of iptables/nftables.

Traffic Splitting Principle

Splitting Criteria

dae supports traffic splitting based on domain name, source IP, destination IP, source port, destination port, TCP/UDP, IPv4/IPv6, process name, MAC address, and other factors.

Among these, source IP, destination IP, source port, destination port, TCP/UDP, IPv4/IPv6, and MAC address can be obtained by parsing MACv2 frames.

The **process name** is obtained by monitoring local process socket, connect, and sendmsg system calls in the cgroupv2 mount point. It then reads and parses the command line from the process control block. This method is significantly faster than user-space programs like Clash that scan the entire procfs to obtain process information (the latter might take even tens of milliseconds).

The **domain name** is obtained by intercepting DNS requests and associating the requested domain name with the corresponding IP address. However, this method has some potential issues:

1. It might lead to misjudgment. For example, if a domestic and a foreign website sharing the same IP address are accessed simultaneously within a short period, or if

the browser employs DNS caching.

2. The user's DNS requests must traverse dae. This can be achieved by setting dae as the DNS server or using a public DNS while dae serves as the gateway.

Despite these challenges, this approach is already an optimal solution compared to other methods. For instance, the Fake IP approach cannot perform IP-based splitting and is plagued by severe cache pollution issues. Similarly, domain sniffing can only intercept traffic like TLS/HTTP. While SNI sniffing for traffic splitting is effective, eBPF's limitations on program complexity and its lack of support for loops prevent us from implementing domain sniffing in the kernel space.

Hence, if DNS requests cannot pass through dae, domain-based splitting will not succeed.

To mitigate DNS pollution and achieve improved CDN connection speeds, dae employs domain sniffing in user space. When `dial_mode` is set to "domain" or its variants and proxied traffic needs to be processed, dae sends the sniffed domain to the proxy server instead of sending the IP address. Consequently, the proxy server re-resolves the domain and connects using the optimal IP. This approach addresses DNS pollution and enhances CDN connection speed.

Additionally, advanced users who have used alternative splitting solutions and don't wish to route DNS requests through dae but still want certain traffic to be split based on domain (e.g., splitting traffic to Netflix nodes and download nodes based on the target domain, with some directly connecting via the core) can enforce the use of sniffed domains for splitting by setting `dial_mode: domain++`.

dae achieves traffic splitting by redirecting traffic using the program in the tc mount point. The redirection is based on the splitting result, either redirecting the traffic to dae's tproxy port or allowing it to bypass dae and go directly.

Proxy Mechanism

The proxy mechanism of dae is akin to other programs. However, when binding to the LAN interface, dae leverages eBPF to directly associate the socket buffer of the traffic to be proxied in the tc mount point with the socket of dae's tproxy listening port. While binding to the WAN interface, dae transfers the socket buffer of the traffic to be proxied from the egress queue of the network card to the ingress queue. It also disables checksums and modifies the destination address to the tproxy listening port.

In terms of benchmarking, dae's proxy performance slightly surpasses that of other proxy programs, but the difference is not significant.

Direct Connection Mechanism

Conventionally, traffic splitting involves passing traffic through a proxy program, navigating the splitting module, and then determining whether to use a proxy or establish a direct connection. This process requires parsing, processing, and copying traffic through the network stack, delivering it to the proxy program, and subsequently copying, processing, and encapsulating it through the network stack before sending it out. This consumes substantial resources. Particularly in scenarios like BitTorrent downloads, even if a direct connection is set, it still consumes numerous connections, ports, memory, and CPU resources. It might even impact NAT type in gaming situations due to the proxy program's inadequate handling, resulting in connection errors.

dae performs traffic splitting at an earlier kernel stage, forwarding directly connected traffic through layer 3 routing. This approach reduces overhead by minimizing transitions between kernel and user space. At this point, Linux functions as a pure switch or router.

For effective direct connection, advanced users with specific network topologies should ensure that, after configuring the kernel parameters and **disabling** dae, other devices can access the network normally when the device with dae is set as the gateway. For instance, accessing 223.5.5.5 should yield a "UrlPathError" response. When performing tcpdump on the dae-equipped device, request packets from client devices should be visible.

Consequently, dae doesn't perform SNAT for directly connected traffic. In setups with a "side-router," this leads to asymmetric routing. In this scenario, when sent out, traffic from client devices passes through dae to the gateway, but when received, traffic goes directly from the gateway to client devices, bypassing dae.

Here, "side-router" refers to: 1) functioning as the gateway, 2) performing SNAT on TCP/UDP, and 3) having the LAN and WAN interfaces in the same network segment.

For example, if a laptop is at 192.168.0.3, the side-router is at 192.168.0.2, and the router is at 192.168.0.1, the logical three-layer topology would be: laptop -> side-router -> router. On the router side, only TCP/UDP traffic with a source IP of 192.168.0.2 would be visible, and no TCP/UDP traffic with a source IP of 192.168.0.3 would be present.

To our knowledge, we are the pioneers of this "side-router" definition (laughter).

Asymmetric routing brings an advantage and a potential issue:

1. It can enhance performance. Since return traffic doesn't traverse dae, direct connection performance becomes as swift as without a side-router, reducing the path.
2. It might disrupt stateful firewall's state maintenance and lead to packet loss (e.g., Sophos Firewall). However, this issue generally doesn't occur in home networks.

From a benchmark perspective, dae's direct connectivity performance is formidable compared to other proxy solutions.

Is the grammar refined?

Yes, the grammar has been refined in the provided text. It reads more smoothly and maintains the technical accuracy of the content. If you have any further questions or need additional adjustments, feel free to ask!